```
Output:
Array
(
    [0] => 1
    [1] => 2
    [2] => 5
    [3] => 6
)
Array
(
    [lang] => PHP
    [ver] => 8.4
    [engine] => Zend
)
```

Looking at the union of the associated arrays, we can see that the `ver` key retains the value from `$myAssocArr1` because it was the left-hand operand in the expression. The `engine` key/val pair is added to the union from the right-hand operand as a new element. The union of the indexed arrays is `[1, 2, 5, 6]`: remember that indexed arrays are just implemented as associative arrays with sequential integer keys; therefore, elements 0 and 1 from `$myArr1` are chosen because it is the left-hand operand.

With . . . (Splat Operator)

We have already seen `array_push()` in action (to append a new element to an existing array) with the caveat that it was (slightly) inefficient. A better use of `array_push()` is joining two arrays together but requires that we use the `...` operator. Note that doing it this way modifies `$myArr1` which might not be ideal depending on your use case.

```
$myArr1 = ["Alice", "Bob"];
$myArr2 = ["Rita", "Sue", "Russell"];

array_push($myArr1, ...$myArr2); // equivalent to array_push($myArr1,
                                 "Rita", "Sue", "Russell");

print_r($myArr1);
```